

High-Level Design: Enterprise Knowledge Graph Query System

Version: 1.0 Date: 2025-11-26 Status: Draft

Table of Contents

1. [System Overview](#)
2. [Architecture Diagrams](#)
3. [Component Breakdown](#)
4. [Data Flow](#)
5. [Technology Stack](#)
6. [Data Models & Schemas](#)
7. [API Contracts](#)
8. [Caching Strategy](#)
9. [Error Handling](#)
10. [Performance Considerations](#)
11. [Future Enhancements](#)

1. System Overview

1.1 Purpose

An enterprise knowledge graph query system that processes natural language queries by extracting relevant entities, exploring graph neighborhoods, and generating answers with reasoning using multi-sample LLM inference and adjudication.

1.2 Scope

This HLD covers the **inference/query processing pipeline**. The database building stage will be addressed separately.

1.3 Key Requirements

- **Query Volume:** ~40 queries/user/day (20 batch + 20 miscellaneous)
- **Response Time:** ~1 minute per query
- **Answer Quality:** Multiple reasoning paths (n=8) with adjudication
- **Caching:** Entity-specific page extract caching for repeat queries
- **Disambiguation:** Handle entity ambiguity with user interaction

2. Architecture Diagrams

2.1 High-Level System Architecture

```

graph TB
    subgraph "User Layer"
        UI[User Interface]
    end

    subgraph "Orchestration Layer"
        QO[Query Orchestrator<br/>FastAPI]
    end

    subgraph "Processing Services"
        EE[Entity Extraction<br/>LLM + NER]
        ER[Entity Resolution<br/>PostgreSQL]
        GE[Graph Exploration<br/>Neo4j]
        DR[Document Retrieval<br/>S3 + Redis Cache]
        SC[Sufficiency Checker<br/>Gemini Flash Lite]
        AG[Answer Generation<br/>Gemini/GPT n=8]
        ADJ[Adjudicator<br/>Gemini/GPT]
    end

    subgraph "Data Layer"
        SQL[(PostgreSQL<br/>Entity Metadata)]
        NEO[(Neo4j<br/>Knowledge Graph)]
        S3[(S3<br/>Documents)]
        REDIS[(Redis<br/>Cache)]
    end

    subgraph "Fallback Services"
        FB[Llama 3<br/>AWS Lambda]
    end

    UI --> QO
    QO --> EE
    EE --> ER
    ER --> SQL
    ER --> GE
    GE --> NEO
    GE --> DR
    DR --> S3
    DR --> REDIS
    DR --> SC
    SC -->|Insufficient| GE
    SC -->|Sufficient| AG
    AG -->|Error| FB
    AG --> ADJ
    ADJ --> QO
    QO --> UI

    style UI fill:#e1f5ff
    style QO fill:#fff4e1
    style EE fill:#e8f5e9
    style ER fill:#e8f5e9
    style GE fill:#e8f5e9
    style DR fill:#e8f5e9
    style SC fill:#e8f5e9
    style AG fill:#e8f5e9
    style ADJ fill:#e8f5e9
    style FB fill:#ffebee
    style SQL fill:#f3e5f5
    style NEO fill:#f3e5f5
    style S3 fill:#f3e5f5
    style REDIS fill:#f3e5f5

```

2.2 Detailed Data Flow Sequence Diagram

```

sequenceDiagram
    participant User
    participant Orchestrator
    participant EntityExtraction
    participant EntityResolution
    participant GraphExplorer
    participant CacheManager
    participant S3Storage
    participant SufficiencyChecker
    participant AnswerGenerator
    participant Adjudicator

    User->>Orchestrator: Submit Query
    Orchestrator->>EntityExtraction: Extract entities/clusters

    alt Batch Query
        EntityExtraction->>EntityExtraction: Use NER
    else Misc Query
        EntityExtraction->>EntityExtraction: Use LLM
    end

    EntityExtraction->>EntityResolution: Entity names
    EntityResolution->>EntityResolution: Query PostgreSQL

    alt Multiple Matches
        EntityResolution->>User: Disambiguation options
        User->>EntityResolution: Select entity
    end

    EntityResolution->>GraphExplorer: Resolved entity IDs
    GraphExplorer->>GraphExplorer: Neo4j: n=2 hops
    GraphExplorer->>CacheManager: Check cache for pages

    alt Cache Miss
        CacheManager->>S3Storage: Fetch document pages
        S3Storage->>CacheManager: Return pages
        CacheManager->>CacheManager: Extract text & cache
    end

    CacheManager->>SufficiencyChecker: Query + Page extracts

    alt Insufficient Context
        SufficiencyChecker->>GraphExplorer: Expand to n=4 hops
        GraphExplorer->>CacheManager: Fetch additional pages
    end

    SufficiencyChecker->>AnswerGenerator: Final context
    AnswerGenerator->>AnswerGenerator: Generate n=8 samples

    alt Primary LLM Fails
        AnswerGenerator->>AnswerGenerator: Trigger Llama 3 fallback
    end

    AnswerGenerator->>Adjudicator: 8 answer candidates
    Adjudicator->>Adjudicator: Evaluate & select best
    Adjudicator->>Orchestrator: Final answer + reasoning
    Orchestrator->>User: Response with citations

```

2.3 Component Interaction Diagram

```

graph LR
    subgraph "Query Type Classification"
        A[User Query] --> B{Query Type?}
        B -->|Batch| C[NER Extraction]
        B -->|Misc| D[LLM Extraction]
    end

    subgraph "Entity Resolution Flow"
        C --> E[Entity Names]
        D --> E
        E --> F{SQL Lookup}
        F -->|Exact Match| G[Unique IDs]
        F -->|Multiple| H[Disambiguation UI]
        F -->|No Match| I[Fuzzy Matching]
        H --> G
        I -->|Found| H
        I -->|Not Found| J[Ask User to Clarify]
    end

    subgraph "Adaptive Graph Exploration"
        G --> K[Neo4j: n=2 hops]
        K --> L[Retrieve Page Metadata]
        L --> M{Cache Hit?}
        M -->|Yes| N[Cached Page Extracts]
        M -->|No| O[Fetch from S3]
        O --> P[Extract Text]
        P --> Q[Store in Cache]
        Q --> N
        N --> R{Sufficient Context?}
        R -->|No| S[Neo4j: n=4 hops]
        S --> L
        R -->|Yes| T[Proceed to Answer Gen]
    end

    subgraph "Answer Generation & Adjudication"
        T --> U[LLM: n=8 samples]
        U -->|Error| V[Fallback: Llama 3]
        U --> W[8 Answer Candidates]
        V --> W
        W --> X[Adjudicator Evaluates]
        X --> Y[Final Answer]
    end

    style A fill:#e1f5ff
    style J fill:#ffebee
    style Y fill:#c8e6c9

```

2.4 Error Handling Flow Diagram

flowchart TD

Start[Query Processing Start] --> EE[Entity Extraction]

EE -->|Success| ER[Entity Resolution]

EE -->|No Entities Found| E1{Fuzzy Match Clusters?}

E1 -->|Yes| ER

E1 -->|No| ERR1[Ask User to Clarify]

ER -->|Unique Match| GE[Graph Exploration]

ER -->|Multiple Matches| E2[Show Disambiguation UI]

E2 --> GE

ER -->|Fuzzy Suggestions| E3[Suggest Alternatives]

E3 -->|User Selects| GE

GE -->|Success| DR[Document Retrieval]

GE -->|Timeout| E4[Reduce to n=1, Retry]

E4 --> DR

DR -->|Success| SC[Sufficiency Check]

DR -->|S3 Failure| E5[Skip Missing Pages]

E5 --> SC

DR -->|Text Extraction Error| E6[Try OCR]

E6 -->|Still Fails| E5

E6 -->|Success| SC

SC -->|Sufficient| AG[Answer Generation]

SC -->|Insufficient & n=2| E7[Expand to n=4]

E7 --> GE

SC -->|Insufficient & n=4| E8[Proceed with Low Confidence]

E8 --> AG

AG -->|Success| ADJ[Adjudication]

AG -->|Primary LLM Error| E9[Trigger Llama 3 Fallback]

E9 -->|Success| ADJ

E9 -->|Failure| E10{Cached Answer?}

E10 -->|Yes| Return[Return Cached]

E10 -->|No| ERR2[Error: Retry Later]

ADJ -->|High Agreement| Success[Return Answer]

ADJ -->|Low Agreement| E11[Return Lesser Evil + Low Confidence]

E11 --> Success

style Start fill:#e1f5ff

style Success fill:#c8e6c9

style ERR1 fill:#ffebee

style ERR2 fill:#ffebee

style E1 fill:#fff9c4

style E2 fill:#fff9c4

style E3 fill:#fff9c4

style E4 fill:#fff9c4

style E5 fill:#fff9c4

style E6 fill:#fff9c4

style E7 fill:#fff9c4

style E8 fill:#fff9c4

style E9 fill:#fff9c4

style E10 fill:#fff9c4

style E11 fill:#fff9c4

2.5 Caching Architecture Diagram

```

graph TB
    subgraph "Cache Layers"
        L1["L1: Entity→Pages Mapping<br/>Key: entity_graph:ID:hops"]
        L2["L2: Page Text Extracts<br/>Key: entity_pages:ID:doc:page"]
        L3["L3: Batch Query Cache<br/>Pre-warmed for CEO queries"]
    end

    subgraph "Cache Operations"
        Q[Query Arrives]
        Q --> C1["C1{Entity in L1?}"]
        C1 -->|Hit| P1[Get Page List]
        C1 -->|Miss| N1[Query Neo4j]
        N1 --> S1[Store in L1]
        S1 --> P1

        P1 --> C2["C2{Pages in L2?}"]
        C2 -->|Hit| R1[Return Cached Text]
        C2 -->|Miss| N2[Fetch from S3]
        N2 --> E1[Extract Text]
        E1 --> S2[Store in L2]
        S2 --> R1
    end

    subgraph "Pre-warming Strategy"
        B[Batch Queries]
        B --> PR[Pre-compute Entities]
        PR --> PW1[Warm L1 Cache]
        PR --> PW2[Warm L2 Cache]
        PW1 --> L3
        PW2 --> L3
    end

    subgraph "Redis Configuration"
        RC[Redis Cluster]
        RC --> TTL[TTL: Forever]
        RC --> EV[Eviction: None]
        RC --> REP[Replication: Enabled]
    end

    L1 --> RC
    L2 --> RC
    L3 --> RC

    style Q fill:#e1f5ff
    style R1 fill:#c8e6c9
    style RC fill:#f3e5f5

```

2.6 End-to-End Processing Timeline

```
gantt
    title Query Processing Timeline (Target: ~60 seconds)
    dateFormat ss
    axisFormat %S s

    section Entity Processing
    Entity Extraction (LLM/NER)      :e1, 00, 5s
    Entity Resolution (SQL)          :e2, after e1, 2s

    section Graph & Retrieval
    Graph Exploration n=2 (Neo4j)    :g1, after e2, 5s
    Document Retrieval (S3+Cache)    :d1, after g1, 8s
    Sufficiency Check                :s1, after d1, 3s

    section Conditional Expansion
    Graph Expansion n=4 (if needed) :g2, after s1, 7s
    Additional Doc Retrieval         :d2, after g2, 5s

    section Answer Generation
    Multi-Sample Generation (n=8)    :a1, after d2, 25s
    Adjudication                     :a2, after a1, 8s

    section Response
    Format & Return                   :r1, after a2, 2s
```

3. Component Breakdown

3.1 Query Orchestrator

Responsibility: Entry point for all queries; manages workflow state machine.

Key Functions:

- Classify query type (batch preset vs miscellaneous)
- Route to appropriate entity extraction method
- Manage retry logic and error propagation
- Track query execution metrics

Technology: Python (FastAPI or similar), State machine library (e.g., `transitions`)

3.2 Entity Extraction Service

Responsibility: Extract entities, clusters, and optionally relation types from user query.

Sub-components:

3.2.1 LLM-based Extractor (Miscellaneous Queries)

- **Model:** Gemini/GPT
- **Input:** User query + DB schema context
- **Output:**

```
{
  "entities": ["Employee X", "Department A"],
  "clusters": ["Engineering Department"],
  "relation_types": ["reports_to", "works_on"]
}
```

3.2.2 NER-based Extractor (Batch Queries)

- **Model:** spaCy, Hugging Face NER models (e.g., `dslim/bert-base-NER`), or Flair
- **Input:** Predefined query templates optimized for NER
- **Output:** Same JSON schema as LLM extractor

Technology:

- LLM: Gemini API / OpenAI API
- NER: spaCy 3.x or Hugging Face Transformers

3.3 Entity Resolution Service

Responsibility: Map extracted entity names to unique IDs in the enterprise database.

Key Functions:

1. **Exact Matching:** Query SQL table for exact entity name matches
2. **Fuzzy Matching:** Use Levenshtein distance or `fuzzywuzzy` if no exact match
3. **Disambiguation UI:**
 - If multiple matches → query enriched SQL table for bio + immediate relations
 - Present options to user for selection
4. **Error Handling:** If no entities found (even fuzzy), return to user with clarification request

Data Store:

- **Primary Table:** `entity_lookup` (`entity_name`, `unique_id`)
- **Enriched Table:** `entity_metadata` (`unique_id`, `bio_data`, `immediate_relations_summary`)

Technology: PostgreSQL (supports fuzzy matching extensions like `pg_trgm`)

3.4 Graph Exploration Service

Responsibility: Traverse knowledge graph to find relevant entities, relations, and associated document pages.

Workflow:

1. Start from resolved entity IDs
2. Execute Cypher query for `n=2` hop traversal
3. Filter by relation types if specified
4. Extract metadata: (`document_id`, `page_number`) from node/edge properties
5. If sufficiency check fails → expand to `n=4` hops

Sample Cypher Query (n=2):

```
MATCH path = (start:Entity)-[*1..2]-(neighbor)
WHERE start.unique_id IN $entity_ids
  AND (size($relation_types) = 0 OR type(relationships(path)[0]) IN $relation_types)
RETURN DISTINCT neighbor.unique_id,
  neighbor.document_id,
  neighbor.page_number,
  type(relationships(path)[0]) AS relation_type
```

Technology: Neo4j (Community or Enterprise Edition)

3.5 Document Retrieval Service

Responsibility: Fetch document pages from object storage and extract text.

Key Functions:

1. Receive list of (`document_id`, `page_number`) tuples from graph exploration
2. Check cache for pre-extracted text (keyed by `entity_id`)
3. If cache miss:
 - Fetch PDF/image from S3
 - Extract text (PyPDF2, pdfplumber, or OCR via Tesseract)
 - Store in cache
4. Return aggregated text extracts

Technology:

- Storage: AWS S3
 - Text Extraction: `pdfplumber`, `PyMuPDF`, or `Tesseract` (OCR)
 - Cache: Redis (see section 8)
-

3.6 Sufficiency Checker

Responsibility: Determine if retrieved context is sufficient to answer the query.

Workflow:

1. Input: User query + `n=2` hop page extracts
2. Prompt LLM: "Can you confidently answer this query with the provided context? Yes/No"
3. If No → trigger `n=4` hop expansion
4. If Yes → proceed to answer generation

Model: Gemini 2.5 Flash Lite (fast, cost-effective)

Sample Prompt:

Given the following question and context, can you confidently provide an accurate answer?

Question: {user_query}

Context: {page_extracts}

Respond with JSON: {"sufficient": true/false, "reason": "..."}

3.7 Answer Generation Service

Responsibility: Generate multiple answer candidates with reasoning.

Configuration:

- **Model:** Gemini Pro or GPT-4 (with fallback to Llama 3 on AWS Lambda)
- **Sampling:** n=8 parallel candidates
- **Temperature:** 0.7
- **Output Format:** JSON with `analysis` and `prediction` fields

System Prompt:

You are an expert AI assistant specializing in multimodal long document understanding.

Output Format:

Your entire response MUST be a single valid JSON object with exactly two fields:

- **analysis:** Briefly explain your thought process. Describe which pages/tables/figures you referenced and how you connected the information to the question.
- **prediction:** Direct answer to the user's question.

User Input:

Question: {user_query}

Evidence:

{aggregated_page_extracts_with_metadata}

API Call (Gemini Example):

```
response = gemini.generate_content(  
    [SYSTEM_PROMPT, user_input],  
    generation_config={  
        'candidate_count': 8,  
        'temperature': 0.7,  
        'response_mime_type': 'application/json'  
    }  
)
```

Fallback Strategy:

- On any error code from primary LLM → trigger Llama 3 hosted on AWS Lambda
- Same n=8 sampling configuration
- Accept lower quality as acceptable fallback

Technology:

- Primary: Gemini API / OpenAI API
 - Fallback: Llama 3 (quantized) on AWS Lambda (with increased timeout/memory)
-

3.8 Adjudicator Service

Responsibility: Select the best answer from 8 candidates based on reasoning quality and citation count.

Input:

```
{
  "query": "...",
  "candidates": [
    {
      "analysis": "I found the answer on pages 3, 7, and 12...",
      "prediction": "The answer is X",
      "citation_count": 3,
      "documents_referenced": 2
    }
  ]
}
```

Evaluation Criteria:

1. Logical consistency of reasoning
2. Number of citations (page count)
3. Diversity of document sources
4. Coherence of prediction

Model: Same as answer generation (Gemini/GPT)

Prompt Strategy:

```
You are an expert adjudicator. Review the 8 answer candidates below and select the
BEST one based on:
- Logical reasoning quality
- Number of supporting citations
- Answer confidence
```

If all answers are poor, return the "lesser evil" with a confidence flag.

Output JSON:

```
{
  "selected_answer": "...",
  "reasoning": "...",
  "confidence": "high/medium/low",
  "citation_count": N
}
```

Technology: Gemini API / OpenAI API

3.9 Response Formatter

Responsibility: Package final answer for end-user or developer debug mode.

Output Modes:

User Mode:

```
{
  "answer": "The CEO should focus on...",
  "citations": [
    {"document_id": "doc_123", "pages": [3, 7, 12]},
    {"document_id": "doc_456", "pages": [5]}
  ],
  "confidence": "high"
}
```

Debug Mode (for developers):

```
{
  "answer": "...",
  "citations": [...],
  "confidence": "high",
  "debug_info": {
    "entities_extracted": ["Employee X", "Dept A"],
    "entities_resolved": [{"name": "Employee X", "id": "emp_001"}],
    "hop_count_used": 2,
    "pages_retrieved": 15,
    "all_8_candidates": [...],
    "adjudication_reasoning": "..."
  }
}
```

4. Data Flow

4.1 End-to-End Query Processing Flow

Stage-by-Stage Breakdown:

STAGE 1: Entity Extraction

- **Input:** Natural language query
- **Process:**
 - Classify query type (batch vs misc)
 - Route to NER (batch) or LLM (misc)
 - Extract entities, clusters, relation types
- **Output:** ["Employee X", "Dept A"], ["Engineering"], ["reports_to"]

STAGE 2: Entity Resolution

- **Input:** Entity names
- **Process:**
 - SQL lookup with exact matching
 - Fuzzy matching if no exact match
 - User disambiguation if multiple matches
- **Output:** ["emp_001", "dept_a"]

STAGE 3: Graph Exploration (n=2)

- **Input:** Resolved entity IDs
- **Process:**
 - Neo4j Cypher 2-hop traversal
 - Extract document metadata from nodes/edges
- **Output:** [(doc_id, page_num, entity_id), ...]

STAGE 4: Document Retrieval

- **Input:** Document/page tuples
- **Process:**
 - Check Redis cache (entity-specific keys)
 - Fetch from S3 if cache miss
 - Extract text and store in cache
- **Output:** {entity_id: [page_texts]}

STAGE 5: Sufficiency Check

- **Input:** Query + page extracts
- **Process:**
 - LLM evaluates if context is sufficient
 - If NO → trigger n=4 expansion (back to Stage 3)
- **Output:** {sufficient: true/false, reason: "..."}

STAGE 6: Answer Generation

- **Input:** Query + final context
- **Process:**
 - LLM generates n=8 samples in parallel
 - Fallback to Llama 3 on error
- **Output:** [{analysis, prediction}, ... × 8]

STAGE 7: Adjudication

- **Input:** 8 answer candidates
- **Process:**
 - Evaluate reasoning quality
 - Count citations
 - Select best answer
- **Output:** {answer, reasoning, confidence, citations}

STAGE 8: Response Formatting

- **Input:** Final answer + metadata
- **Process:**
 - Format for user mode or debug mode
- **Output:** Structured JSON response

5. Technology Stack

Component	Technology	Rationale
Query Orchestrator	Python + FastAPI	Async support, easy integration with AI APIs
Entity Extraction (LLM)	Gemini API / GPT-4 API	Strong NLU, schema understanding
Entity Extraction (NER)	spaCy 3.x or Hugging Face	Fast, pre-trained models available
Entity Resolution DB	PostgreSQL	pg_trgm for fuzzy matching, ACID compliance
Knowledge Graph	Neo4j	Optimized for multi-hop traversals, Cypher expressiveness
Document Storage	AWS S3	Scalable, low-latency object storage
Text Extraction	pdfplumber / PyMuPDF	Robust PDF parsing
Cache	Redis	In-memory, sub-millisecond reads, TTL support
Sufficiency Checker	Gemini 2.5 Flash Lite	Fast, cost-effective
Answer Generation	Gemini Pro / GPT-4	Multi-candidate sampling support
Fallback LLM	Llama 3 (quantized) on AWS Lambda	Self-hosted, cost control
Adjudicator	Gemini Pro / GPT-4	Same as answer generation for consistency
API Gateway	AWS API Gateway or FastAPI	Rate limiting, auth, monitoring
Logging & Monitoring	CloudWatch / Datadog	Centralized logs, performance metrics

6. Data Models & Schemas

6.1 SQL Database Schemas

Entity Lookup Table

```
CREATE TABLE entity_lookup (
    unique_id VARCHAR(50) PRIMARY KEY,
    entity_name VARCHAR(255) NOT NULL,
    entity_type VARCHAR(50), -- e.g., 'employee', 'department', 'project'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_entity_name (entity_name)
);

-- Enable fuzzy matching
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX trgm_idx_entity_name ON entity_lookup USING gin (entity_name gin_trgm_ops);
```

Entity Metadata Table (for Disambiguation)

```
CREATE TABLE entity_metadata (
    unique_id VARCHAR(50) PRIMARY KEY REFERENCES entity_lookup(unique_id),
    bio_data JSON, -- e.g., {"title": "Senior Engineer", "location": "NYC"}
    immediate_relations JSON, -- e.g., [{"type": "reports_to", "entity": "Manager Y"}]
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

6.2 Neo4j Graph Schema

Node Labels

- Entity (employees, departments, projects, etc.)
- Document (optional, for document-level nodes)

Node Properties (Example: Employee Entity)

```
(:Entity {
    unique_id: "emp_001",
    name: "Employee X",
    type: "employee",
    document_id: "doc_123",
    page_numbers: [3, 7, 12],
    metadata: {...}
})
```

Edge Properties (Example: REPORTS_TO Relationship)

```
(emp1:Entity)-[:REPORTS_TO {
    document_id: "doc_456",
    page_number: 5,
    extraction_confidence: 0.95
}]->(emp2:Entity)
```

6.3 Cache Schema (Redis)

Key Structure

```
entity_pages:{entity_id}:{document_id}:{page_number}
```

Value (JSON)

```
{
  "extracted_text": "Full text content from page...",
  "extraction_timestamp": "2025-11-26T10:00:00Z",
  "document_metadata": {
    "title": "Q4 Report",
    "version": "v2.3"
  }
}
```

TTL Policy

- **Forever** (no expiration) for inference cache
- Future: Invalidate on graph updates (not in scope)

6.4 API Request/Response Schemas

Query Request

```
{
  "query": "What are the top 10 priorities for the CEO?",
  "query_type": "batch" | "misc",
  "user_id": "user_123",
  "debug_mode": false
}
```

Query Response (User Mode)

```
{
  "query_id": "q_xyz789",
  "answer": "The top priorities are...",
  "citations": [
    {
      "document_id": "doc_123",
      "document_title": "Strategic Plan 2025",
      "pages": [3, 7, 12]
    }
  ],
  "confidence": "high",
  "processing_time_seconds": 58.3
}
```

Query Response (Debug Mode)

```
{
  "query_id": "q_xyz789",
  "answer": "...",
  "citations": [...],
  "confidence": "high",
  "processing_time_seconds": 58.3,
  "debug_info": {
    "entities_extracted": ["CEO", "Q4 Goals"],
    "entities_resolved": [
      {"name": "CEO", "id": "emp_ceo_001", "match_type": "exact"}
    ],
    "hop_count_used": 2,
    "pages_retrieved": 15,
    "cache_hits": 8,
    "cache_misses": 7,
    "sufficiency_check_result": "sufficient",
    "answer_candidates": [...],
    "adjudication_reasoning": "Selected sample 3 due to highest citation count and clearest logic"
  }
}
```

7. API Contracts

7.1 Internal Service APIs

Entity Extraction Service

```
POST /extract-entities
Body: {
  "query": "string",
  "extraction_method": "llm" | "ner"
}
Response: {
  "entities": ["string"],
  "clusters": ["string"],
  "relation_types": ["string"]
}
```

Entity Resolution Service

```
POST /resolve-entities
Body: {
  "entity_names": ["Employee X", "Dept A"]
}
Response: {
  "resolved": [
    {"name": "Employee X", "unique_id": "emp_001", "match_type": "exact"}
  ],
  "ambiguous": [
    {
      "name": "John Smith",
      "candidates": [
        {"id": "emp_123", "bio": {...}, "relations": [...]},
        {"id": "emp_456", "bio": {...}, "relations": [...]}
      ]
    }
  ],
  "not_found": ["XYZ Corp"]
}
```

Graph Exploration Service

```
POST /explore-graph
Body: {
  "entity_ids": ["emp_001", "dept_a"],
  "hop_count": 2,
  "relation_types": ["reports_to"]
}
Response: {
  "pages_to_retrieve": [
    {"entity_id": "emp_001", "document_id": "doc_123", "page_number": 3}
  ]
}
```

Document Retrieval Service

```
POST /retrieve-pages
Body: {
  "pages": [
    {"document_id": "doc_123", "page_number": 3, "entity_id": "emp_001"}
  ]
}
Response: {
  "page_extracts": {
    "emp_001": ["text from page 3", "text from page 7"]
  },
  "cache_stats": {"hits": 5, "misses": 3}
}
```

Sufficiency Checker

```
POST /check-sufficiency
Body: {
  "query": "string",
  "context": ["page extract 1", "page extract 2"]
}
Response: {
  "sufficient": true | false,
  "reason": "Sufficient information to answer about CEO priorities"
}
```

Answer Generation Service

```
POST /generate-answers
Body: {
  "query": "string",
  "context": ["page extracts"],
  "sample_count": 8,
  "temperature": 0.7
}
Response: {
  "candidates": [
    {"analysis": "...", "prediction": "..."}
  ],
  "model_used": "gemini-pro",
  "fallback_triggered": false
}
```

Adjudicator Service


```
POST /adjudicate
Body: {
  "query": "string",
  "candidates": [
    {"analysis": "...", "prediction": "...", "citation_count": 3}
  ]
}
Response: {
  "selected_answer": "...",
  "reasoning": "...",
  "confidence": "high" | "medium" | "low",
  "selected_candidate_id": 3
}
```

8. Caching Strategy

8.1 Cache Layers

L1: Entity→Pages Mapping Cache

- **Purpose:** Avoid redundant Neo4j queries for same entities
- **Key:** `entity_graph:{entity_id}:hops{n}`
- **Value:** List of `(document_id, page_number)` tuples
- **TTL:** Forever (invalidate on graph rebuild)

L2: Page Extract Cache

- **Purpose:** Avoid re-extracting text from S3 documents
- **Key:** `entity_pages:{entity_id}:{document_id}:{page_number}`
- **Value:** Extracted text + metadata
- **TTL:** Forever
- **Eviction:** None (assume storage is cheap, queries are read-heavy)

8.2 Cache Population Strategy

Batch Query Pre-warming

- For preset batch queries (e.g., "Top 10 CEO priorities"):
 - Pre-compute entity extraction
 - Pre-warm L1 and L2 caches during off-peak hours
 - Store in separate "batch cache" namespace

Miscellaneous Query Reactive Caching

- Cache on first retrieval
- Subsequent queries benefit from cache hits

8.3 Cache Invalidation (Future)

- On knowledge graph update:
 - Invalidate L1 cache for affected entities
 - Mark L2 cache entries for re-validation
- Not in current scope (assume static graph during inference)

8.4 Technology Choice

Redis is ideal for this use case:

- Sub-millisecond read latency
- Supports complex data structures (JSON, lists)
- Optional persistence (RDB snapshots for recovery)
- Horizontal scaling via Redis Cluster

9. Error Handling

9.1 Error Categories & Handling

Error Type	Detection	Handling Strategy	User Feedback
No Entities Extracted	Entity extraction returns empty list	Fuzzy match clusters; if still empty → ask user	"Could not identify relevant entities. Please clarify: Are you asking about [suggestions]?"
Entity Ambiguity	Multiple SQL matches for same entity name	Query enriched metadata table, present options	"Multiple matches found: 1) John Smith (Engineering) 2) John Smith (Sales). Please select."
Fuzzy Match Suggestions	No exact match, but fuzzy score > 0.7	Suggest top 3 matches	"No exact match for 'Jon Smith'. Did you mean: John Smith, Jane Smith?"
Graph Traversal Timeout	Neo4j query exceeds 30s	Reduce hop count to n=1, retry	Log warning, continue with partial results
Document Retrieval Failure	S3 fetch fails (404, 403)	Skip missing pages, log error	Continue with available pages (may trigger insufficiency)
Text Extraction Error	PDF parsing crashes	Fall back to OCR; if OCR fails → skip page	Log error, reduce citation count for that entity
LLM API Failure	429, 500, timeout from Gemini/GPT	Trigger Llama 3 fallback	Transparent to user (logged in debug mode)
Fallback LLM Failure	Llama 3 also fails	Return cached answer if available; else error	"Unable to process query. Please try again later."
Insufficiency at n=4	Sufficiency check still fails at max hops	Proceed with answer generation anyway	Flag in debug mode; lower confidence score
Adjudicator Indecision	All 8 samples have conflicting answers	Return "lesser evil" + low confidence flag	"Answer confidence is low. Consider rephrasing query."

9.2 Retry Logic

Transient Errors (Network, Rate Limits)

```
@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=2, max=10),
    retry=retry_if_exception_type((Timeout, ConnectionError, HTTPError))
)
def call_llm_api(prompt):
    # API call logic
    pass
```

Permanent Errors (400, 404)

- No retry
- Log error with full context
- Trigger fallback or error response immediately

9.3 Circuit Breaker Pattern

For external services (Gemini API, Neo4j):

- **Threshold:** 5 consecutive failures
- **Action:** Open circuit → switch to fallback
- **Reset:** After 60s of success

9.4 Graceful Degradation

- Missing pages → Continue with partial context
- LLM failure → Use fallback model
- Cache miss → Fetch from source (slower, but functional)

10. Performance Considerations

10.1 Latency Budget (Target: ~60 seconds)

Stage	Estimated Time	Optimization Strategy
Entity Extraction	2-5s	Use NER for batch queries (faster than LLM)
Entity Resolution	1-2s	Index SQL tables on entity_name; use pg_trgm
Graph Exploration (n=2)	3-5s	Optimize Cypher queries; limit result size
Document Retrieval	5-10s	Cache aggressively; parallel S3 fetches
Sufficiency Check	2-3s	Use fastest model (Flash Lite)
Graph Expansion (n=4, if needed)	5-10s	Same as n=2 exploration
Answer Generation (n=8)	20-30s	Parallel API calls; use batching if supported
Adjudication	5-10s	Single LLM call with all candidates
Total	45-70s	Within acceptable range

10.2 Optimization Techniques

Parallelization

- **S3 Fetches:** Use `asyncio` or thread pool for parallel page retrieval
- **LLM Sampling:** Gemini/GPT natively support `candidate_count=8` in single call
- **Cache Lookups:** Batch Redis GET operations using `mget`

Database Query Optimization

- **Neo4j:**
 - Use parameterized queries
 - Add indexes on `unique_id` properties
 - Limit traversal depth (cap at n=4)
- **PostgreSQL:**
 - Use `EXPLAIN ANALYZE` to optimize fuzzy matching queries
 - Maintain materialized views for enriched entity metadata

LLM Call Optimization

- **Prompt Engineering:** Keep prompts concise to reduce token count
- **Model Selection:** Use Flash models for sufficiency check
- **Batch Processing:** For preset queries, pre-compute answers and cache

Caching Wins

- **Cache Hit Scenarios:**
 - Batch queries (high overlap in entities)
 - Follow-up queries from same user (temporal locality)
- **Expected Hit Rate:** 60-70% for batch queries, 30-40% for misc queries

10.3 Scalability Considerations

Horizontal Scaling

- **Stateless Services:** All components (extraction, retrieval, generation) are stateless
- **Load Balancing:** Use AWS ALB or NGINX for query orchestrator
- **Neo4j Clustering:** Use Neo4j Causal Cluster for read scaling

Vertical Scaling

- **Lambda for Fallback LLM:** Auto-scales based on invocations
- **Redis:** Use Redis Cluster for memory scaling

Cost Optimization

- **LLM Costs:** Primary cost driver
 - Use cheaper models for sufficiency check (Flash Lite)
 - Cache answers for batch queries (avoid re-generation)
 - Set rate limits to prevent runaway costs
 - **S3 Costs:** Use S3 Intelligent-Tiering for infrequently accessed docs
-

11. Future Enhancements

11.1 Planned Improvements

Phase 2: Advanced Answer Quality

- **Multi-modal Evidence:** Include tables, charts, images in answer generation
- **Cross-document Reasoning:** Synthesize answers spanning multiple documents
- **Confidence Scoring:** ML model to predict answer reliability

Phase 3: Interactive Refinement

- **Follow-up Questions:** Allow users to drill down on answers
- **Clarification Loop:** System proactively asks for disambiguation before processing
- **Answer Explanation:** Visualize evidence trail (which pages → reasoning → answer)

Phase 4: Knowledge Graph Updates

- **Incremental Updates:** Real-time graph updates with cache invalidation
- **Version Control:** Track knowledge graph versions, allow querying historical state
- **Conflict Resolution:** Handle contradictory information across documents

11.2 Monitoring & Analytics

Metrics to Track

- **Query Metrics:**
 - Average response time per stage
 - Cache hit rate (L1, L2)
 - Entity resolution success rate
 - Sufficiency check pass rate (n=2 vs n=4)
- **Quality Metrics:**
 - Answer confidence distribution
 - User satisfaction ratings (if available)
 - Adjudicator agreement rate (how often top candidate is selected)
- **Cost Metrics:**
 - LLM API costs per query
 - S3 data transfer costs
 - Redis cache memory usage

Logging Strategy

- **Structured Logging:** JSON format with trace IDs
- **Log Levels:**
 - DEBUG: Cache hits/misses, entity resolution steps
 - INFO: Query start/end, stage transitions
 - WARN: Fallback triggers, partial results
 - ERROR: API failures, data retrieval errors

Alerting

- **Critical Alerts:**
 - LLM API failure rate > 10%
 - Average response time > 90s
 - Neo4j query timeout rate > 5%
 - **Warning Alerts:**
 - Cache hit rate < 50% (for batch queries)
 - Entity resolution ambiguity rate > 20%
-

12. Example Workflows

Example 1: Batch Query (CEO Dashboard)

Query: "What are the top 10 priorities for Q1 2025?"

Flow:

1. **Entity Extraction (NER):** Entities: ["Q1 2025", "priorities"], Clusters: ["strategic goals"]
2. **Entity Resolution:** Resolves "Q1 2025" → project_q1_2025, "priorities" → strategic_goals_cluster
3. **Graph Exploration (n=2):** Finds 12 document pages linked to these entities
4. **Document Retrieval:** Cache HIT for 8/12 pages (pre-warmed batch cache)
5. **Sufficiency Check:** "Sufficient" (skips n=4)
6. **Answer Generation:** 8 samples generated, citation counts: [3, 5, 4, 5, 3, 6, 4, 5]
7. **Adjudication:** Selects sample 6 (highest citation count, clear reasoning)
8. **Response:** Returns answer with 6 document citations, confidence: high

Example 2: Miscellaneous Query (Ambiguous Entity)

Query: "Who does John report to?"

Flow:

1. **Entity Extraction (LLM):** Entities: ["John"], Relation types: ["reports_to"]
2. **Entity Resolution:** Multiple matches found (3 employees named John)
3. **Disambiguation:** System presents options:
 - o John Smith (Engineering, emp_123)
 - o John Doe (Sales, emp_456)
 - o John Lee (Marketing, emp_789)
4. **User Selection:** User selects "John Smith"
5. **Graph Exploration (n=2):** Finds 3 pages with "reports_to" relations
6. **Sufficiency Check:** "Sufficient"
7. **Answer Generation:** 8 samples, all converge on "Reports to Jane Doe (Manager)"
8. **Adjudication:** High agreement, selects sample 2
9. **Response:** Clear answer with 3 citations, confidence: high

Example 3: Insufficient Context (Expands to n=4)

Query: "What was the outcome of the Project Phoenix retrospective?"

Flow:

1. **Entity Extraction (LLM):** Entities: ["Project Phoenix"], Clusters: ["retrospective"]
2. **Entity Resolution:** Exact match → project_phoenix_id
3. **Graph Exploration (n=2):** Finds only 2 pages (meeting invite, agenda)
4. **Sufficiency Check:** "Insufficient - no outcome details found"
5. **Graph Expansion (n=4):** Finds 8 additional pages (meeting notes, action items)
6. **Sufficiency Check:** "Sufficient"
7. **Answer Generation:** 8 samples generated
8. **Adjudication:** Selects answer citing "3 key action items from retrospective notes"
9. **Response:** Detailed answer with 5 citations, confidence: medium

Appendix: Glossary

Term	Definition
Entity	A specific person, project, department, or concept in the knowledge graph
Cluster	A grouping/category of entities (e.g., "Engineering Department", "Q1 Projects")
Hop	One degree of separation in graph traversal (n=2 means entities 2 edges away)
Sufficiency Check	LLM evaluation of whether retrieved context can answer the query
Adjudication	Process of selecting the best answer from multiple candidates
Batch Query	Pre-defined recurring query with optimized processing (e.g., CEO dashboards)
Miscellaneous Query	Ad-hoc user query requiring full LLM-based entity extraction
Page Extract	Text content extracted from a specific document page
Citation	Reference to source document and page number supporting an answer

Document Control

Version	Date	Author	Changes
1.0	2025-11-26	System Architect	Initial HLD with visual diagrams

END OF DOCUMENT